4 DSP

4.1 DSP Watchdog Timer

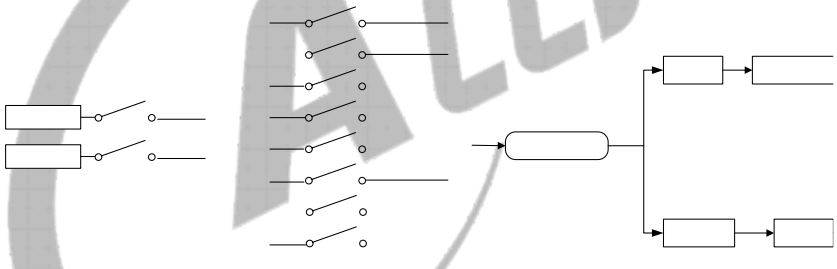
4.1.1 Overview

The DSP watchdog timer (DSP WDT) is used to transmit a reset signal to reset the entire system when an exception occurs in the system. The main features for the watchdog are as follows:

- Supports 12 initial values
- Supports the generation of timeout interrupts
- Supports the generation of reset signals
- Supports Watchdog Restart

4.1.2 Block Diagram

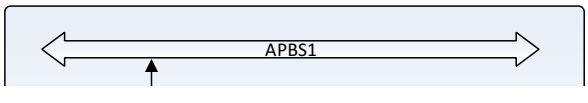
Figure 4- 1 DSP WDT Block Diagram



4.1.3 Functional Description

4.1.3.1 Typical Application

Figure 4- 2 DSP WDT Typical Application



4.1.3.2 Operationg Modes

The watchdog has two operating modes: the interrupt mode and reset mode.

- In the interrupt mode, when the counter value reaches 0 and RWDOG_IRQ_EN_REG is enabled, the watchdog generates an interrupt.
- In the reset mode, when the counter value reaches 0, the watchdog generates a reset signal to reset the entire system.

You can configure the operating mode for the watchdog via the bit[1:0] of the RWDOG_CFG_REG. The value 0x2 is for the interrupt mode and the value 0x1 is for the reset mode.

Both the interrupt mode and reset mode support Watchdog Restart. You can make the watchdog to count from the initial value at any time by configuring the RWDOG_CTRL_REG: write 0xA57 to bit[12:1], then write 1 to bit[0].

4.1.3.3 Initializing the Watchdog

Follow the steps below to initialize the watchdog:

1. Write the bit[1:0] of RWDOG_CFG_REG to configure the watchdog operating mode so that the watchdog can generate interrupts or output reset signals.
2. Write the bit[7:4] of RWDOG_MODE_REG to configure the initial count value.
3. 3Write the bit[0] of RWDOG_MODE_REG to enable the watchdog.

4.1.3.4 Processing the Interrupt

In the interrupt mode, the watchdog is used as a counter. It generates an interrupt everytime the count value reaches 0.

Follow the steps below to process the interrupt:

1. Write the enable bit of RWDOG_IRQ_EN_REG to enable the interrupt.
2. After entering the interrupt process, write the pending bit of RWDOG_IRQ_STA_REG to clear the interrupt pending and execute the process of waiting for the interrupt.
3. Resume the interrupt and continue to execute the interrupted process.

4.1.4 Register List

Module Name	Base Address	Comments
DSP_Watchdog	0x0170 1000	DSP Watchdog configuration register

Register Name	Offset	Description
RWDOG_IRQ_EN_REG	0x0000	DSP_Watchdog IRQ Enable Register
RWDOG_IRQ_STA_REG	0x0004	DSP_Watchdog Status Register
RWDOG_CTRL_REG	0x0010	DSP_Watchdog Control Register
RWDOG_CFG_REG	0x0014	DSP_Watchdog Configuration Register
RWDOG_MODE_REG	0x0018	DSP_Watchdog Mode Register
RWDOG_OUTPUT_CFG_REG	0x001C	DSP_Watchdog Output Config Register

4.1.5 Register Description

4.1.5.1 0x0000 DSP_Watchdog IRQ Enable Register (Default Value: 0x0000_0000)

Offset:0x0000			Register Name: RWDOG_IRQ_EN_REG
Bit	Read/Write	Default/Hex	Description
31:1	/	/	/
0	R/W	0x0	RWDOG_IRQ_EN. DSP_Watchdog Interrupt Enable. 0: DSP_Watchdog interrupt mask, 1: DSP_Watchdog interrupt enable.

4.1.5.2 0x0004 DSP_Watchdog Status Register (Default Value: 0x0000_0000)

Offset:0x0004			Register Name: RWDOG_IRQ_STA_REG
Bit	Read/Write	Default/Hex	Description
31:1	/	/	/
0	R/W	0x0	RWDOG_IRQ_PEND. DSP_Watchdog IRQ Pending, Set 1 to the bit will clear it. 0: No effect, 1: Pending, DSP_Watchdog interval value is reached.

4.1.5.3 0x0010 DSP_Watchdog Control Register (Default Value: 0x0000_0000)

Offset:0x0010			Register Name: RWDOG_CTRL_REG
Bit	Read/Write	Default/Hex	Description
31:13	/	/	/
12:1	W	0x0	RWDOG_KEY_FIELD. DSP_Watchdog Key Field. Should be written at value 0xA57. Writing any other value in this field aborts the write operation.
0	R/W	0x0	RWDOG_RESTART. DSP_Watchdog Restart. 0: No effect,

Offset:0x0010			Register Name: RWDOG_CTRL_REG
Bit	Read/Write	Default/Hex	Description
			1: Restart the DSP_Watchdog.

4.1.5.4 0x0014 DSP_Watchdog Configuration Register (Default Value: 0x0000_0001)

Offset:0x0014			Register Name: RWDOG_CFG_REG
Bit	Read/Write	Default/Hex	Description
31:16	W	0x0	KEY_FIELD. Key Field. This field should be filled with 0x16AA, and then the bit 0 can be written with the new value.
15:7	/	/	/
8	R/W	0x0	RWDOG_CLK_SRC. Select DSP_Watchdog clock sources. 0: HOSC_32K 1: LOSC_32K
7:2	/	/	/
1:0	R/W	0x1	DSP_WDOG_CONFIG. DSP_Watchdog generates a reset signal 00:/ 01: to whole system 10: only interrupt 11: /

4.1.5.5 0x0018 DSP_Watchdog Mode Register (Default Value: 0x0000_0000)

Offset:0x0018			Register Name: RWDOG_MODE_REG
Bit	Read/Write	Default/Hex	Description
31:16	W	0x0	KEY_FIELD. Key Field. This field should be filled with 0x16AA, and then the bit 0 can be written with the new value.
15:8	/	/	/
7:4	R/W	0x0	DSP_WDOG_INTV_VALUE. DSP_Watchdog Interval Value. When the clock source is HOSC_32K, the time of 16000 cycles is equal to 0.5 sec, and when the clock source is 32K, the time of 16000 cycles is based on the number of 32K. 0000: 16000 cycles 0001: 32000 cycles 0010: 64000 cycles 0011: 96000 cycles 0100: 128000 cycles 0101: 160000 cycles

Offset:0x0018			Register Name: RWDOG_MODE_REG
Bit	**Read/Write**	**Default/Hex**	**Description**
			0110: 192000 cycles 0111: 256000 cycles 1000: 320000 cycles 1001: 384000 cycles 1010: 448000 cycles 1011: 512000 cycles 1100: / 1101: / 1110: / 1111: /
3:1	/	/	/
0	R/W	0x0	DSP_WDOG_EN. DSP_Watchdog Enable. 0: No effect; 1: Enable the DSP_Watchdog.

4.1.5.6 0x001C DSP_Watchdog Output Config Register (Default Value: 0x0000_001F)

Offset: 0x001C			Register Name:RWDOG_OUTPUT_CFG_REG
Bit	**Read/Write**	**Default/Hex**	**Description**
31:12	/	/	/
11:0	R/W	0x1F	DSP_Watchdog_Output_Config DSP_Watchdog rst valid time config T=1/32ms*(N+1) Default 1ms

4.2 DSP Interrupt List

4.2.1 DSP_CORE Interrupt Source

BInterrupt Pin	Interrupt Name	Priority	Bit In Source	Description
0	nmi	nmi	136	External Non-Mask Interrupt
	timer.1	3		DSP inter timer1 Interrupt
	timer.0	3		DSP inter timer0 Interrupt
1	DSP_Msgbox	3		DSP_SYS Massage box read irq
2	Msgbox0	3	85	CPUX Massage box 0 irq(for DSP/RISCV)
3	Msgbox1	3	128	RISCV Message box 0 irq(for DSP/CPU)
4	Dmac_irq_dsp_s	3	127	Dmac 8~15 channel secure irq
	sw	3		DSP inter software Interrupt
5	Dmac_irq_dsp_ns	3	126	Dmac 8~15 channel non-secure irq
6	GIC_OUT_CO	3	160	GIC_OUT_CO cpucfg interrupt
7	I2S0	2	26	I2S0 irq
8	I2S1	2	27	I2S1 irq
9	I2S2	2	28	I2S2 irq
10	AUDIO_Codec	2	25	AUDIO codec irq
11	DMIC	2	24	DMIC irq
12	Spinlock	2	54	SPLOCK irq
13	DSP_WDOG	2	122	DSP_SYS WDOG interrupt.
14	DSP_TZMA	2	125	DSP_SYS TZMA interrupt
15	HSTIMER0	2	55	
16	HSTIMER1	2	56	
17	INTC	1		DSP_SYS int ctrl module
18	TIMER0	1	59	
19	TIMER1	1	60	
20	GPA	1	57	
21	LR	1	61	
22	TPA	1	62	
23	M_TIMER0	1	140	
24	M_TIMER1	1	141	
25	M_TIMER2	1	142	
26	M_TIMER3	1	143	
27	M_ALARM0	1	144	
	writeerr	1		DSPAXI Response Write Error

4.2.2 DSP_INTC Interrupts

BInterrupt Pin	Interrupt Name	Priority	Bit In Source	Description
0	NA		0	Not Used
			1	Not Used
			2	Not Used
1	UART0		3	SYS uart0 interrupt
2	UART1		4	SYS uart1 interrupt
3	UART2		5	SYS uart2 interrupt
4	UART3		6	SYS uart3 interrupt
5	UART4		7	SYS uart4 interrupt
6	UART5		8	SYS uart5 interrupt
			9	Not Used
7	TWI0		10	SYS twi0 interrupt
8	TWI1		11	SYS twi1 interrupt
9	TWI2		12	SYS twi2 interrupt
10	TWI3		13	SYS twi3 interrupt
			14	Not Used
			15	Not Used
11	SPI0		16	SYS spi0 interrupt
12	SPI1		17	SYS spi1 interrupt
			18	Not Used
13	PWM		19	SYS pwm interrupt
14	IR_TX		20	SYS irrx interrupt
15	LEDC		21	LED control interrupt
16	CAN0		22	CAN0 interrupt
17	CAN1		23	CAN1 interrupt
	SPDIF		24	SYS spdif interrupt audio
	DMIC		25	DMIC interrupt
	AUDIO_CODEC		26	AUDIO CODEC interrupt
	I2S_PCM0		27	SYS i2s0 interrupt
	I2S_PCM1		28	SYS i2s1 interrupt
	I2S_PCM2		29	SYS i2s2 interrupt
18	USB0_DEVICE		30	SYS usb0 interrupt
19	USB0_EHCI		31	SYS usb0_ehci interrupt
20	USB0_OHCI		32	SYS usb0_ohci interrupt
			33	Not Used
21	USB1_EHCI		34	SYS usb1_ehci interrupt
22	USB1_OHCI		35	SYS usb1_ohci interrupt
			36	Not Used
			37	Not Used
			38	Not Used
			39	Not Used

BInterrupt Pin	Interrupt Name	Priority	Bit In Source	Description
			40	Not Used
23	SD0		41	SYS sd0 interrupt
24	SD1		42	SYS sd1 interrupt
25	SD2		43	SYS sd2 interrupt
26	MSI		44	SYS msi interrupt
27	SMC		45	SYS smc interrupt
			46	Not Used
28	GMAC		47	SYS gmac0 interrupt
29	TZMA_ERR		48	TZMA_ERR interrrupt
30	CCU_FERR		49	SYS ccu err interrupt
31	AHB_HREADY_TIME_OUT		50	SYS hready timeout interrupt
32	DMA_NS		51	Dmac 0~7 channel non-secure irq
33	DMA_S		52	Dmac 0~7 channel secure irq
34	CE_NS		53	CE irq, (ce_irq_ns)
35	CE_S		54	CE irq, (ce_irq_s)
	SPINLOCK		55	SYS splock interrupt AUDIO
	HSTIMER0		56	SYSCPUS hstimer 0 interrupt
	HSTIMER1		57	SYSCPUS hstimer 1 interrupt
	GPA		58	SYSCPUS timer 0 interrupt
36	THS		59	SYSCPUS timer 1 interrupt
	TIMER0		60	GPADC_IRQ
	TIMER1		61	THS_IRQ
	LR		62	LRADC_IRQ
	TPA		63	TPADC_IRQ
37	WATCHDOG		64	SYS watchdog interrupt
38	IOMMU		65	IOMMU interrupt
			66	Not Used
39	VE		67	VE interrupt
			68	Not Used
			69	Not Used
40	GPIOB_NS		70	SYS gpiob_ns interrupt
41	GPIOB_S		71	SYS gpiob_s interrupt
42	GPIOC_NS		72	SYS gpioc_ns interrupt
43	GPIOC_S		73	SYS gpioc_s interrupt
44	GPIOD_NS		74	SYS gpiod_ns interrupt
45	GPIOD_S		75	SYS gpiod_s interrupt
46	GPIOE_NS		76	SYS gpioe_ns interrupt
47	GPIOE_S		77	SYS gpioe_s interrupt
48	GPIOF_NS		78	SYS gpiof_ns interrupt
49	GPIOF_S		79	SYS gpiof_s interrupt

BInterrupt Pin	Interrupt Name	Priority	Bit In Source	Description
50	GPIOG_NS		80	SYS gpiog_ns interrupt
51	GPIOG_S		81	SYS gpiog_s interrupt
			82	Not Used
			83	Not Used
			84	Not Used
			85	Not Used
52	CPUX_MSGBOX_DSP_W		86	CPUX MSGBOX WRITE IRQ FOR DSP
			87	
53	DE		88	DE interrupt
54	DI		89	DI interrupt
55	G2D		90	G2D interrupt
56	LCD		91	LCD interrupt
57	TV		92	TV interrupt
58	DSI		93	DSI interrupt
59	HDMI		94	HDMI interrupt
60	TVE		95	TVE interrupt
61	CSI_DMA0		96	CSI_DMA0 interrupt
62	CSI_DMA1		97	CSI_DMA1 interrupt
			98	CSI_DMA2 interrupt
			99	CSI_DMA3 interrupt
			100	Not Used
63	CSI_PARSER0		101	CSI_PARSER0 interrupt
			102	Not Used
			103	Not Used
			104	Not Used
			105	Not Used
			106	Not Used
64	CSI_TOP_PKT		107	CSI_TOP_PKT interrupt
65	TVD		108	TVD interrupt
			109	Not Used
			110	Not Used
			111	Not Used
			112	Not Used
			113	Not Used
			114	Not Used
			115	Not Used
			116	Not Used
			117	Not Used
			118	Not Used
			119	Not Used

BInterrupt Pin	Interrupt Name	Priority	Bit In Source	Description
			120	Not Used
66	DSP_DEE		121	DSP DoubleExceptionError interrupt
67	DSP_PFE		122	DSP PFatalError interrupt
68	DSP_WDG		123	DSP wdg interrupt
69	DSP_MBOX_CPUX_W		124	DSP to CPUX mbox interrupt
70	DSP_MBOX_RISCV_W		125	DSP to RISCV mbox interrupt
71	DSP_TZMA		126	DSP TZMA interrupt
	DMAC_IRQ_DSP_NS		127	Dmac 8~15 channel non-secure irq
	DMAC_IRQ_DSP_S		128	Dmac 8~15 channel secure irq
72	RISCV_MBOX_RISCV		129	RISCV MSGBOX READ IRQ FOR RISCV
73	RISCV_MBOX_DSP		130	RISCV MSGBOX WRITE IRQ FOR DSP
74	RISCV_MBOX_CPUX		131	RISCV MSGBOX WRITE IRQ FOR RISCV
75	RISCV_WDG		132	RISCV WatchDog IRQ
76	RISCV_SYS4		133	RISCV No Defination
77	RISCV_SYS5		134	RISCV No Defination
78	RISCV_SYS6		135	RISCV No Defination
79	RISCV_SYS7		136	RISCV No Defination
80	NMI		137	CPUS NMI interrupt
81	PPU		138	CPUS PPU interrupt
82	TWD		139	CPUS TWD interrupt
			140	
83	TIMER0		141	CPUS TIMER0 IRQ
84	TIMER1		142	CPUS TIMER1 IRQ
85	TIMER2		143	CPUS TIMER2 IRQ
86	TIMER3		144	CPUS TIMER3 IRQ
87	ALARM0		145	CPUS ALARM0 IRQ
			146	
			147	
			148	
			149	
			150	
			151	
88	IRRX		152	CPUS IRRX_IRQ[9:0] XOR to 1bit
			153	

BInterrupt Pin	Interrupt Name	Priority	Bit In Source	Description
			154	
89	AHBS_HREADY_TOUT		155	CPUS AHB READY TIME OUT IRQ
			156	
			157	
			158	
			159	
			160	
90	GIC_OUT_CO		161	

4.3 DSP Message Box

4.3.1 Overview

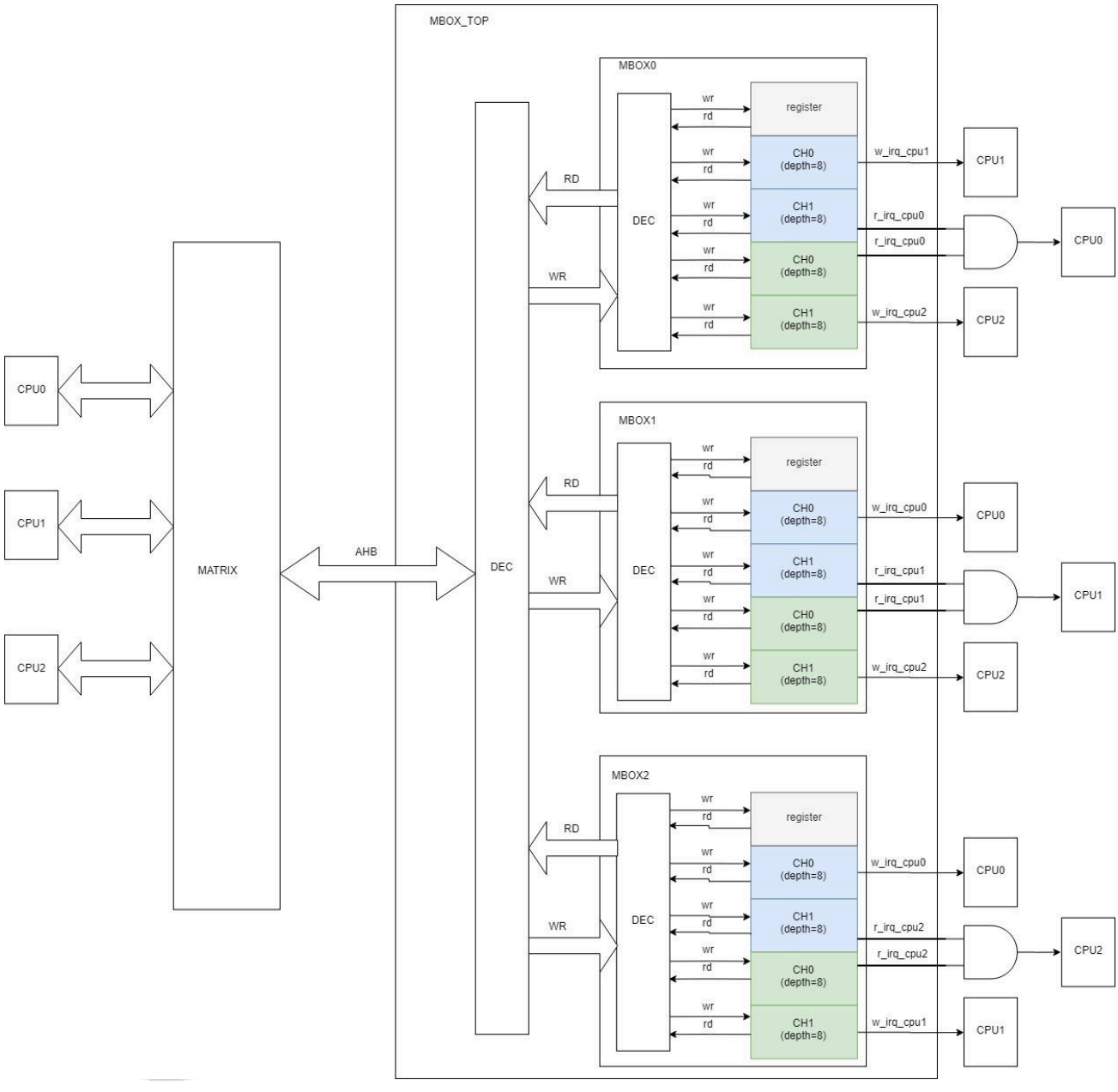
The DSP Message Box (DSP MSGBOX) provides interrupt communication mechanism for on-chip processor.

The MSGBOX has the following features:

- Supports maximum 4 CPUs to transmit information through channels. Each CPU has a MSGBOX. The number of CPU can be flexibly configured based on needs. This section will take 3 CPUs as an example:
  - CPU 0: ARM CPUX
  - CPU 1: DSP
  - CPU2: RISC-V
- The channels of each CPU can be flexibly configured (this section will take 2 channels as an eaxmple), and the FIFO depth of a channel is 8 x 32 bits
- Supports interrupts

4.3.2 Block Diagram

Figure 4- 3 DSP MSGBOX Block Diagram



4.3.3 Functional Description

4.3.3.1 Clock and Reset

The MSGBOX is mounted on AHB. Before accessing the MSGBOX registers, you need to de-assert the MSGBOX reset signal on AHB bus and then open the MSGBOX gating signal on AHB bus.

4.3.3.2 Typical Application

Several masters can build communication by configuring the MSGBOX. The communication parties have at least two dua channels. During the communication process, the current status can be judged through the interrupt or FIFO status.

4.3.3.3 Transmitter/Receiver Mode

At the same channel, user1 is fixed as transmitter, user0 is fixed as receiver.

4.3.3.4 Interrupt

Each channel can configure indepedently the interrupt enable bit, a read interrupt will be generated when the channel is empty, a write interrupt will be generated when the channel is non-full. For each CPU, all channels generate a read interrupt together, that is, if only a channel is non-full, the read interrupt will be generated, this channel can be obtained by quering the interrupt status register.

4.3.3.5 FIFO Status

When channel FIFO is non-full, the FIFO_FULL_FLAG is 0, at the moment the FIFO can be written.

When channel FIFO is full, the FIFO_FULL_FLAG is 1, at the moment if FIFO is written again, the first data of FIFO can be covered.

See **MSGBOX_MSG_STATUS_REG** for FIFO status.

4.3.4 Programming Guidelines

4.3.4.1 Checking the Transfer Status via the Interrupt

Follow the steps below to check the transfer status:

Step 1 Enable the interrupt for the channel: Configure the interrupt enable bits of transmitter/receiver through **MSGBOX_WR_IRQ_EN_REG**/**MSGBOX_RD_IRQ_EN_REG**. (user0: RX interrupt enable; user1: TX interrupt enable)

Step 2 Check the IRQ status of the corresponding queue through **MSGBOX_WR_IRQ_STATUS_REG** **or** **MSGBOX_RD_IRQ_STATUS_REG**.

- If the FIFO is not full, the channel generates a transmission interrupt to remind the transmitter to transmit data. Write data to the FIFO in the interrupt handler, then clear the pending bit of the transmitter in **MSGBOX_WR_IRQ_STATUS_REG** and the enable bit of the transmitter in **MSGBOX_WR_IRQ_EN_REG**.

- If the FIFO has new data, the channel generates a reception interrupt to remind the receiver to receive data. Read data from the FIFO in interrupt handler, then clear the pending bit of the receiver in MSGBOX_RD_IRQ_STATUS_REG and the enable bit of the receiver in MSGBOX_RD_IRQ_EN_REG.

4.3.4.2 Checking the Transfer Status via the FIFO

Follow the steps below to check the FIFO status of the corresponding queue:

- If the FIFO is not full, the transmitter fills the FIFO to 8*32 bits.
- If the FIFO is full, the receiver reads the FIFO data, and reads MSGBOX_FIFO_STATUS_REG to acquire the current FIFO data amount and the FIFO data amount before reading, which means no data is dropped.

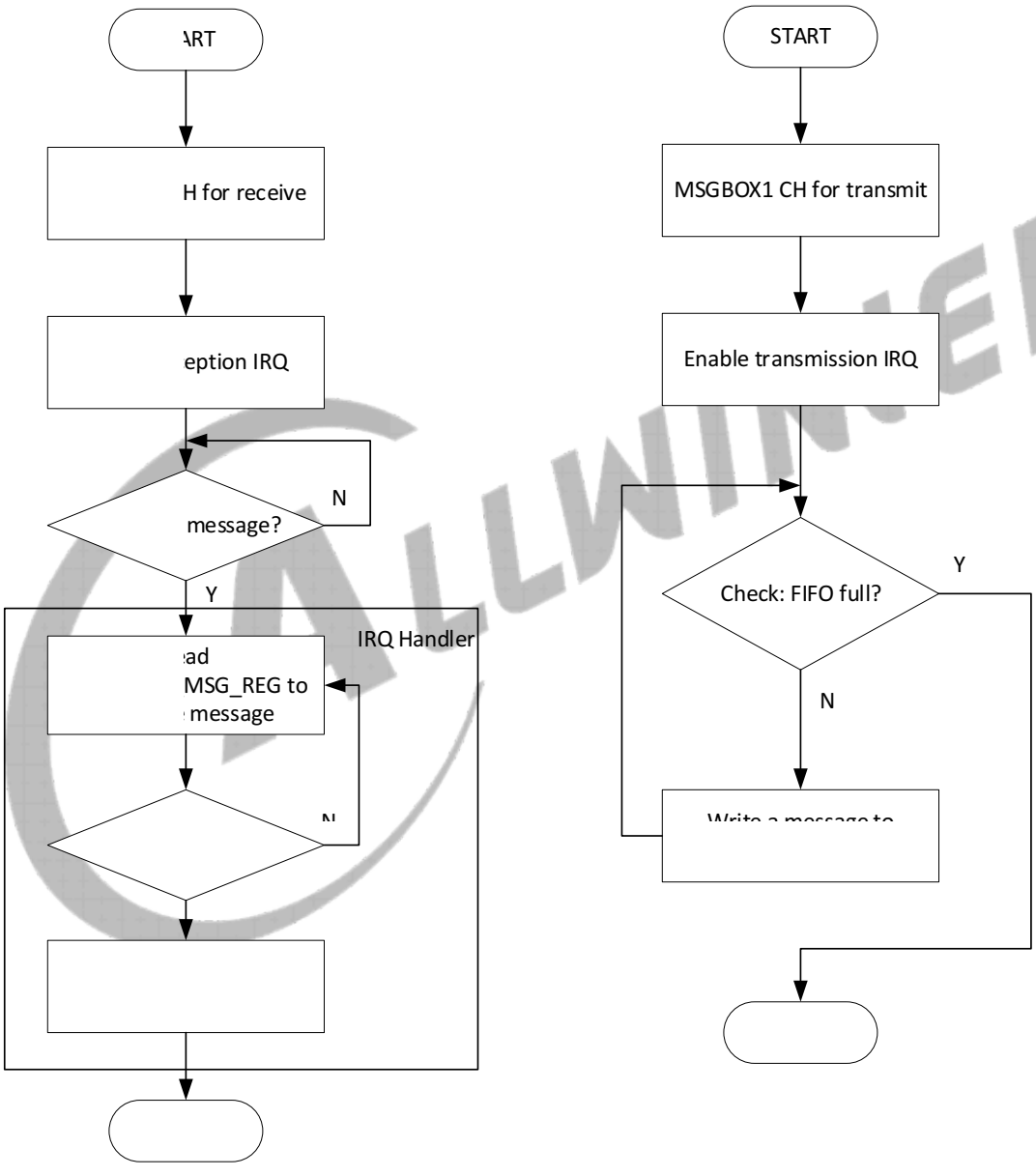
4.3.4.3 Transmitting/Receiving Message

Taking CPU0 as an example, the following figure shows the communication process between MSGBOX0 and MSGBOX1.

MSGBOX0: receiving message

MSGBOX1: transmitting message

Figure 4-4 The Communication Process between MSGBOX0 and MSGBOX1



4.3.5 Register List

Parameter	Description
M	MBOX Num(0-3)

Parameter	Description
N	Dual channel Num in every MBox(0-3)
P	2(Dual channel, one for sync, one for async)

Module Name	Base Address	Comments
DSP_MSGBOX	0x0170_1000	DSP MSGBOX configuration register

Register Name	Offset	Description
MSGBOX_RD_IRQ_EN_REG	0x0020+M*0x400+N*0x100	MSGBox Read IRQ Enable Register
MSGBOX_RD_IRQ_STATUS_REG	0x0024+M*0x400+N*0x100	MSGBox Read IRQ Status Register
MSGBOX_WR_IRQ_EN_REG	0x0030+M*0x400+N*0x100	MSGBox Write IRQ Enable Register
MSGBOX_WR_IRQ_STATUS_REG	0x0034+M*0x400+N*0x100	MSGBox Write IRQ Status Register
MSGBOX_DEBUG_REG	0x0040+M*0x400+N*0x100	MSGBox Debug Register
MSGBOX_FIFO_STATUS_REG	0x0050+M*0x400+N*0x100 +P*0x0004(P=0~1)	MSGBox FIFO Status Register
MSGBOX_MSG_STATUS_REG	0x0060+M*0x400+N*0x100 +P*0x0004(P=0~1)	MSGBox Message Status Register m
MSGBOX_MSG_REG	0x0070+M*0x400+N*0x100 +P*0x0004(P=0~1)	MSGBox Message Queue Register

4.3.6 Register Description

4.3.6.1 MSGBox Read IRQ Enable Register (Default Value:0x00000000)

Offset: 0x0020+M*0x400+N*0x100			Register Name: MSGBOX_RD_IRQ_EN_REG
Bit	**R/W**	**Default/Hex**	**Description**
31:3	/	/	/
2	R/W	0x0	RECEPTION_MQ1_IRQ_EN. Reception MQ1 Interrupt Enable 0: Disable 1: Enable (It will notify user 0 by interrupt when Message Queue 1 has received a new message.)
1	/	/	/
0	R/W	0x0	RECEPTION_MQ0_IRQ_EN. Reception MQ0 Interrupt Enable 0: Disable 1: Enable (It will notify user 0 by interrupt when Message Queue 0 has received a new message.)

4.3.6.2 MSGBox Read IRQ Status Register (Default Value:0x00000000)

Offset: 0x0024+M*0x400+N*0x100			Register Name: MSGBOX_RD_IRQ_STATUS_REG
Bit	**R/W**	**Default/Hex**	**Description**
31:3	/	/	/
2	R/W	0x0	RECEPTION_MQ1_IRQ_PEND. RECEPTION MQ1 IRQ PENDING 0: No effect, 1: Pending. This bit will be pending for user 0 when Message Queue 1 has received a new message. Set one to this bit will clear it.
1	/	/	/
0	R/W	0x0	RECEPTION_MQ0_IRQ_PEND. RECEPTION MQ0 IRQ PENDING 0: No effect, 1: Pending. This bit will be pending for user 0 when Message Queue 0 has received a new message. Set one to this bit will clear it.

4.3.6.3 MSGBox Write IRQ Enable Register (Default Value:0x00000000)

Offset: 0x0030+M*0x400+N*0x100			Register Name: MSGBOX_WR_IRQ_EN_REG
Bit	**R/W**	**Default/Hex**	**Description**
31:4	/	/	/
3	R/W	0x0	TRANSMIT_MQ1_IRQ_EN. TRANSMIT MQ1 IRQ Enable 0: Disable 1: Enable (It will Notify user 1 by interrupt when Message Queue 1 is not full.)
2	/	/	/
1	R/W	0x0	TRANSMIT_MQ0_IRQ_EN. TRANSMIT MQ0 IRQ Enable 0: Disable 1: Enable (It will Notify user 1 by interrupt when Message Queue 0 is not full.)
0	/	/	/

4.3.6.4 MSGBox Write IRQ Status Register (Default Value:0x0000000A)

Offset: 0x0034+M*0x400+N*0x100			Register Name: MSGBOX_WR_IRQ_STATUS_REG
Bit	**R/W**	**Default/Hex**	**Description**
31:16	/	/	/
3	R/W	0x1	TRANSMIT_MQ1_IRQ_PEND. TRANSMIT MQ1 IRQ Pending 0: No effect, 1: Pending. This bit will be pending for user 1 when Message Queue 1 is not full. Set one to this bit will clear it.
2	/	/	/
1	R/W	01	TRANSMIT_MQ0_IRQ_PEND. TRANSMIT MQ0 IRQ Pending 0: No effect, 1: Pending. This bit will be pending for user 1 when Message Queue 0 is not full. Set one to this bit will clear it.
0	/	/	/

4.3.6.5 MSGBox Debug Register (Default Value:0x00000000)

Offset: 0x0040+M*0x400+N*0x100			Register Name: MSGBOX_DEBUG_REG
Bit	**R/W**	**Default/Hex**	**Description**
31:16	/	/	/
15:8	R/W	0x0	FIFO_CTRL. MQ[7:0] Control. In the debug mode, the corresponding FIFO channel will disable and only one register space valid for a message exchange. 0: Normal Mode. 1: Disable the corresponding FIFO (Clear FIFO).
7:1	/	/	/
0	R/W	0x0	DEBUG_MODE. In the Debug Mode, each user can transmit messages to itself through each Message Queue. 0: Normal Mode 1: Debug Mode.

4.3.6.6 MSGBox FIFO Status Register (Default Value:0x00000000)

Offset:0x0050+M*0x400+N*0x100+P*0x0004(P=0~1)			Register Name: MSGBOX_FIFO_STATUS_REG
Bit	**R/W**	**Default/Hex**	**Description**
31: 1	/	/	/
0	R	0x0	FIFO_FULL_FLAG. FIFO FULL FLAG 0: The Message FIFO queue is not full (space is available), 1: The Message FIFO queue is full. This FIFO status register has the status related to the message queue.

4.3.6.7 MSGBox Message Status Register m (Default Value:0x00000000)

Offset:0x0060+M*0x400+N*0x100+P*0x0004(P=0~1)			Register Name: MSGBOX_MSG_STATUS_REG
Bit	**R/W**	**Default/Hex**	**Description**
31:4	/	/	/
3:0	R	0x0	MSG_NUM. Number of unread messages in the message queue. Here, limited to eight messages per message queue. 0000: There is no message in the message FIFO queue. 0001: There is 1 message in the message FIFO queue. 0010: There are 2 messages in the message FIFO queue. 0011: There are 3 messages in the message FIFO queue. 0100: There are 4 messages in the message FIFO queue. 0101: There are 5 messages in the message FIFO queue. 0110: There are 6 messages in the message FIFO queue. 0111: There are 7 messages in the message FIFO queue. 1000: There are 8 messages in the message FIFO queue. 1001~1111:/

4.3.6.8 MSGBox Message Queue Register (Default Value:0x00000000)

Offset:0x0070+M*0x400+N*0x100+P*0x0004(P=0~1)			Register Name: MSGBOX_MSG_REG
Bit	**R/W**	**Default/Hex**	**Description**
31:0	R/W	0x0	The message register stores the next to be read message of the message FIFO queue. Reads remove the message from the FIFO queue.